DSP OpenAMP

使用指南

版本号: 1.3

发布日期: 2023.06.15

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
1.0	2021.12.31	AWA1402	初始版本
1.1	2022.07.19	AWA1903	1. 增加 Linux 管理 DSP 的生命周期章节。 2.Demo 程序中增加简单例子。
1.2	2023.03.17	AWA1903	1. 增加 DSP 启动方式章节。 2. 增加 DSP 的 GPIO 中断互斥配置章节。 3. 增加 rpmsg_test 章节。
1.3	2023.06.15	AWA1903	将 RPbuf 使用指南整合到此文档中。

广州星翼电子科技有限公司zdyzlinux

目　录

1 概述 **1**
1.1 编写目的 1
1.2 适用范围 1
1.3 相关人员 1

2 DSP 启动方式 **2**
2.1 Linux 管理 DSP 的生命周期 2
2.2 uboot 启动 dsp 的配置 3
2.2.1 分区和加载配置 3
2.2.2 DSP 的 menuconfig 配置 4

3 DSP 的 GPIO 中断互斥配置 **5**
3.1 DSP 的 menuconfig 配置 5
3.2 Linux 的 kernel_menuconfig 配置 5
3.3 uboot 的 menuconfig 配置 6

4 使用 RPMsg 进行核间通信 **8**
4.1 如何编写一个 OpenAMP RPMsg 应用程序 8
4.2 API 说明 9
4.2.1 获取 RPMsg Device 9
4.2.2 创建 endpoint 9
4.2.3 等待 endpoint 准备好 10
4.2.4 向远端发送数据 11
4.2.5 销毁 endpoint 12
4.3 Demo 程序 12
4.3.1 rpmsg 驱动层通信程序 12
4.3.1.1 编译 12
4.3.1.2 基本使用 12
4.3.1.3 简单例子 13
4.3.2 rpmsg 应用层通信程序（rpmsg_test） 14
4.3.2.1 编译 14
4.3.2.2 基本使用 14
4.3.2.3 简单例子 15

5 使用 RPBuf 进行核间通信 **16**
5.1 RPBuf 介绍 16
5.2 RPBuf 传输速率 16
5.2.1 Linux 收 DSP 发 16
5.3 RPBuf 相关概念与流程 17

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5.3.1 RPBuf 框架基本组件 17
5.3.2 申请/释放 buffer 18
5.3.3 数据传输流程 22
5.3.4 数据同步 23
5.3.4.1 在 RTOS 中设置同步 23
5.3.4.2 在 linux 用户空间中设置同步 24
5.3.4.3 数据同步流程 24
5.3.5 异步通知 24
5.4 代码分析 25
5.4.1 Linux 25
5.4.1.1 在内核空间使用 RPBuf 25
5.4.1.2 在用户空间使用 RPBuf 31
5.4.2 FreeRTOS 32
5.5 rpbuf_demo 程序 33
5.5.1 代码路径 33
5.5.1.1 linux 33
5.5.1.2 FreeRTOS 34
5.5.2 编译 34
5.5.3 基本使用 34
5.5.4 rpbuf_test 程序 36
5.5.4.1 代码路径 36
5.5.4.2 编译 36
5.5.4.3 基本使用 37

广州星翼电子科技有限公司zdyzlinux

插　图

图 2-1　DSP 的 menuconfig 配置 4
图 3-1　DSP 的中断互斥 menuconfig 配置 5
图 3-2　Linux 的中断互斥 menuconfig 配置 5
图 3-3　uboot 的中断互斥 menuconfig 配置 6
图 5-1　RPBuf Components 17
图 5-2　RPBuf Allocate Buffer Flowchart 19
图 5-3　RPBuf Free Buffer Flowchart 20
图 5-4　RPBuf Buffer States 21
图 5-5　RPBuf Wrok 22
图 5-6　RPBuf Ack Flowchart 24

广州星翼电子科技有限公司zdyzlinux

1 概述

1.1 编写目的

基于 R329_EVB5_PRO_V1 介绍如何使用全志 DSP 的 OpenAMP 框架，OpenAMP 框架主要提供处理器之间的生命周期管理和通信，由于 R329 中的 DSP 核运行 freeRTOS，A53 核运行 Linux，因此本文中也有涉及 Linux 的内容。有一些章节虽然与 OpenAMP 框架没有密切关联，但考虑到这些章节也是 DSP 与 A53 核协调工作时需要用到的，所以本文档也会介绍 DSP 的启动配置和中断互斥配置。同时也会介绍如何使用 RPMsg 和 RPBuf 进行数据传输。

本文描述的内容主要在于 “如何使用” 上，关于 DSP OpenAMP 的设计和实现细节，请参考《DSP_OpenAMP_ 设计文档》。

1.2 适用范围

有 DSP 并且使用 OpenAMP 进行跨核通信的芯片平台，如 R329、R528。

1.3 相关人员

DSP AMP 框架的使用人员。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 DSP 启动方式

2.1 Linux 管理 DSP 的生命周期

DSP 以.bin文件的形式存在于 Linux 的/lib/firmware目录下，通过 remoteproc 驱动来对 DSP 的生命周期进行管理，默认情况下，内核会加载 remoteproc 驱动，在其过程中完成启动 DSP 的操作。

假如我们希望不在内核中自动起 DSP，可以在 dts 中添加no-auto-boot;字段。

```
  dsp0_rproc: dsp0_rproc@0 {
      ......
+      no-auto-boot;
      status = "okay";
  };

  dsp1_rproc: dsp1_rproc@0 {
      ......
+      no-auto-boot;
      status = "okay";
  };
```

然后通过手动输入命令来加载。

启动 dsp 命令：

```
echo start > /sys/kernel/debug/remoteproc/remoteproc0/state
echo start > /sys/kernel/debug/remoteproc/remoteproc1/state
```

关闭 dsp 命令：

```
echo stop > /sys/kernel/debug/remoteproc/remoteproc0/state
echo stop > /sys/kernel/debug/remoteproc/remoteproc1/state
```

通常内核会自动从以下路径中找到固件并加载：

```
/lib/firmware
/lib/firmware/updates
```

我们可以自定义 DSP 固件存放路径，以下命令的含义是告诉内核新增/tmp作为固件的存放路径，然后调用命令启动 dsp0：

```
echo -n /tmp > /sys/module/firmware_class/parameters/path
echo start > /sys/kernel/debug/remoteproc/remoteproc0/state
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.2 uboot 启动 dsp 的配置

因 SDK 中默认让 DSP 在内核中启动，如果有 uboot 启动 DSP 的需求，请参考此章节进行配置。

2.2.1 分区和加载配置

进入/device/config/chips/r329/configs/evb5_pro_v1/linux 目录或 cconfig 命令进入，修改路径下有两个文件 sys_partition.fex 和 env.cfg。

1. sys_partition.fex 中添加 dsp0 和 dsp1 的的分区，内容如下：

```
diff --git a/configs/evb5_pro_v1/linux/sys_partition.fex b/configs/evb5_pro_v1/linux/sys_partition.fex
index 29873b5..c6d0fdb 100644
--- a/configs/evb5_pro_v1/linux/sys_partition.fex
+++ b/configs/evb5_pro_v1/linux/sys_partition.fex
@@ -66,6 +66,18 @@ size = 252
    downloadfile = "rootfs.fex"
    user_type    = 0x8000

+[partition]
+    name       = dsp0
+    size       = 2520
+    downloadfile = "dsp0.fex"
+    user_type    = 0x8000
+
+[partition]
+    name       = dsp1
+    size       = 2520
+    downloadfile = "dsp1.fex"
+    user_type    = 0x8000
+
 [partition]
    name       = recovery
    size       = 28224
```

技巧

其中 size 以 504 对齐，根据实际的 dsp 文件大小调整。

2. env.cfg 文件中添加加载顺序，内容如下：

```
diff --git a/configs/evb5_pro_v1/linux/env.cfg b/configs/evb5_pro_v1/linux/env.cfg
index 9d4d1df..ad3ff98 100644
--- a/configs/evb5_pro_v1/linux/env.cfg
+++ b/configs/evb5_pro_v1/linux/env.cfg
@@ -22,6 +22,8 @@ mtd_name=sys
 root_partition=rootfs
 rootfstype=squashfs
 boot_partition=boot
+dsp0_partition=dsp0
+dsp1_partition=dsp1
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
……
+boot_dsp0=sunxi_flash read 45000000 ${dsp0_partition};bootr 45000000 0 0
+boot_dsp1=sunxi_flash read 45000000 ${dsp1_partition};bootr 45000000 0 1
 boot_normal=sunxi_flash read 45000000 ${boot_partition};bootm 45000000
 boot_recovery=sunxi_flash read 45000000 recovery;bootm 45000000
 boot_fastboot=fastboot
@@ -45,6 +49,6 @@ fastboot_key_value_min=0x2
 bootdelay=0
 #default bootcmd, will change at runtime according to key press
 #default nand boot
-bootcmd=run setargs_nand boot_normal
+bootcmd=run setargs_nand boot_dsp0 boot_dsp1 boot_normal
 #nor boot
 #bootcmd=run setargs_nand boot_normal
```

2.2.2 DSP 的 menuconfig 配置

因 DSP 在 uboot 阶段启动，考虑到 DSP 有些逻辑是需要在 Linux 启动时才执行的，因此在 DSP 中的 make menuconfig 中选上CONFIG_COMPONENTS_OPENAMP_EARLY_BOOT这个选项，具体如下：

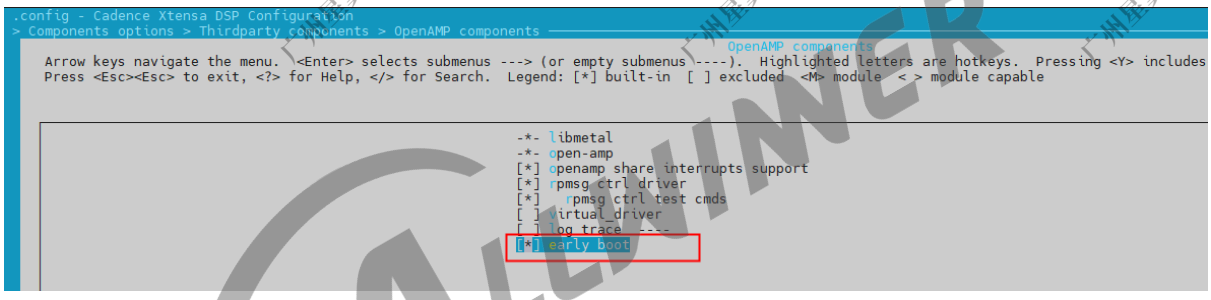

图 2-1: DSP 的 menuconfig 配置

⚠ 注意

Linux 的 remoteproc 驱动在启动 DSP 的过程中，会初始化双方通信使用的共享资源，如果在配置了 DSP 在 uboot 启动后，还需要使用跨核通信功能的话，那么还需要把 DSP 固件放到/lib/firmware目录下（当然 SDK 在编译时会自动把 cbin 目录下的两个 DSP 固件拷到/lib/firmware目录下），在内核启动过程中让 remoteproc 驱动初始化双方通信使用的共享资源，而 CONFIG_COMPONENTS_OPENAMP_EARLY_BOOT 这个配置就是为了 DSP 能正确加载通信使用的共享资源。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3 DSP 的 GPIO 中断互斥配置

对于 R329 而言，DSP 和 A53 都能访问 GPIO 资源，如果有在 DSP 配置 GPIO 中断的需求，请参考此章节。此章节针对以下的一个场景：DSP 在 uboot 启动后配置了相关的 GPIO 中断，然后下一步内核启动，在 GPIO 驱动加载的过程中对所有的 GPIO 中断寄存器进行初始化，此时 DSP 的 GPIO 中断便会失效，对于此场景我们增加了一个 GPIO 中断互斥配置。

3.1 DSP 的 menuconfig 配置

DSP 中的 make menuconfig 中选上CONFIG_COMPONENTS_OPENAMP_SHARE_IRQ这个选项，具体如下：

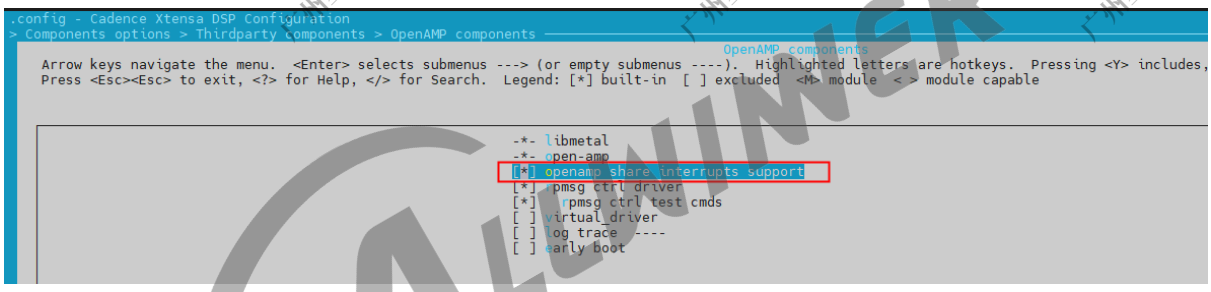

图 3-1: DSP 的中断互斥 menuconfig 配置

3.2 Linux 的 kernel_menuconfig 配置

Linux 中的 make kernel_menuconfig 中选上CONFIG_SUNXI_RPROC_SHARE_IRQ这个选项，具体如下：

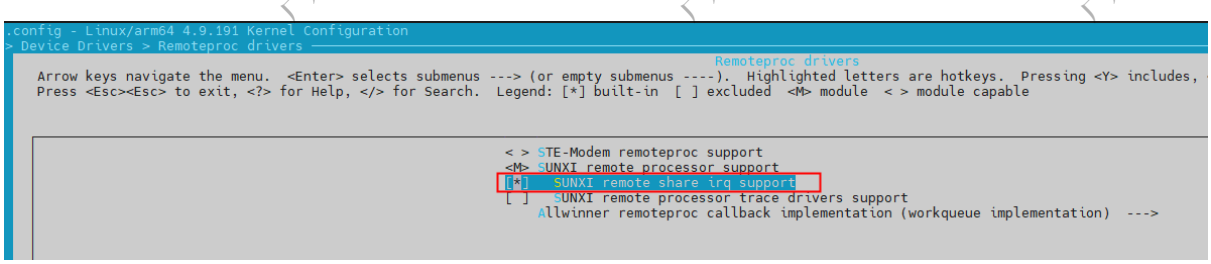

图 3-2: Linux 的中断互斥 menuconfig 配置

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.3 uboot 的 menuconfig 配置

如果 DSP 是配置成 uboot 启动的话，那么 uboot 中的 make menuconfig 也要选上DSP_SUPPORT_SHARE_IRQ 这个选项，具体如下：

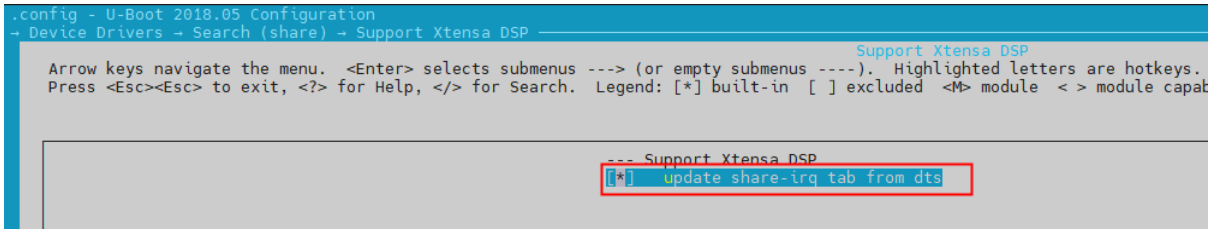

图 3-3: uboot 的中断互斥 menuconfig 配置

下面用一个例子说明如何进行配置。

假设我们整组 GPIOB 给 DSP0 使用，整组 GPIOL 给 DSP1 使用，dts 的配置如下：

```
reserved-irq {
  share-dsp0 {
    arch-name = "dsp0";
    memory-region = <&dsp0_share_irq_table>;
    share-irq =
      <GPIOB 0xffffffff>,<GPIOC 0x00000000>,
      <GPIOF 0x00000000>,<GPIOG 0x00000000>,
      <GPIOH 0x00000000>,<GPIOL 0x00000000>,
      <GPIOM 0x00000000>,<GPION 0x00000000>;
  };

  share-dsp1 {
    arch-name = "dsp1";
    memory-region = <&dsp1_share_irq_table>;
    share-irq =
      <GPIOB 0x00000000>,<GPIOC 0x00000000>,
      <GPIOF 0x00000000>,<GPIOG 0x00000000>,
      <GPIOH 0x00000000>,<GPIOL 0xffffffff>,
      <GPIOM 0x00000000>,<GPION 0x00000000>;
  };
};
```

假设我们将 PG0~7, PF3 给 DSP0 使用，将 PN8~15，PM6 给 DSP1 使用，dts 的配置如下：

```
reserved-irq {
  share-dsp0 {
    arch-name = "dsp0";
    memory-region = <&dsp0_share_irq_table>;
    share-irq =
      <GPIOB 0x00000000>,<GPIOC 0x00000000>,
      <GPIOF 0x00000008>,<GPIOG 0x000000ff>,
      <GPIOH 0x00000000>,<GPIOL 0x00000000>,
      <GPIOM 0x00000000>,<GPION 0x00000000>;
  };

  share-dsp1 {
    arch-name = "dsp1";
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
        memory-region = <&dsp1_share_irq_table>;
        share-irq =
            <GPIOB 0x00000000>,<GPIOC 0x00000000>,
            <GPIOF 0x00000000>,<GPIOG 0x00000000>,
            <GPIOH 0x00000000>,<GPIOL 0x00000000>,
            <GPIOM 0x00000040>,<GPION 0x0000ff00>;
    };
};
```

⚠ 注意

如无特殊需求，建议配置整组 GPIO 给某个 DSP 使用，这样对于系统来说判断效率更高。

广州星翼电子科技有限公司zdyzlinux

4 使用 RPMsg 进行核间通信

RPMsg 框架用于异构的处理器之间进行相互通信。OpenAMP 内部实现了可用于 RTOS 或裸机系统中的 RPMsg 框架，与 Linux 内核的 RPMsg 框架兼容。

4.1 如何编写一个 OpenAMP RPMsg 应用程序

在 OpenAMP 中使用 RPMsg 进行通信的常见步骤如下（为了便于说明，忽略对异常情况的处理）：

```
#include <openamp/rpmsg.h>
#include <openamp/sunxi_helper/openamp_platform.h>

/* Endpoint callback, called when the endpoint receives data. */
static int rpmsg_ept_callback(struct rpmsg_endpoint *ept, void *data,
        size_t len, uint32_t src, void *priv)
{
    /* ... */

    return RPMSG_SUCCESS;
}

/* Endpoint service unbind callback, called when remote endpoint is destroyed. */
static void rpmsg_ns_unbind_callback(struct rpmsg_endpoint *ept)
{
    /* ... */
}

int rpmsg_demo(int argc, char *argv[])
{
    struct rpmsg_endpoint ept;
    struct rpmsg_device *rpmsg_dev;

    /* 1. Get rpmsg device */
    rpmsg_dev = openamp_platform_get_rpmsg_vdev(RPMSG_DEV_ID);

    /* 2. Create endpoint */
    rpmsg_create_ept(&ept, rpmsg_dev, EPT_NAME, EPT_SRC_ADDR, EPT_DST_ADDR,
            rpmsg_ept_callback, rpmsg_ns_unbind_callback);

    /* 3. Waiting for endpoint ready */
    while (! is_rpmsg_ept_ready(ept)) {
        /* sleep... */
    }

    /* 4. Send data to remote */
    rpmsg_send(ept, data, data_len);
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    /* 5. Destroy endpoint */
    rpmsg_destroy_ept(ept);

    return 0;
}
```

1. 获取 RPMsg device，为后面创建 endpoint 做准备。
2. 创建 endpoint。应用程序通过操作 endpoint 与远端处理器进行通信，endpoint 自身绑定着之间通过源地址与目的地址进行区分。当某个 endpoint 收到远端处理器发过来的消息时，它的回调函数会被调用到。
3. 在第一次使用某个 endpoint 向远端发送消息前，需要等待它准备好。
4. 使用 rpmsg_send 向远端发送信息。
5. 当不再需要使用 endpoint 时，使用 rpmsg_destroy_ept 销毁它。

4.2 API 说明

4.2.1 获取 RPMsg Device

```
/* <openamp/sunxi_helper/openamp_platform.h> */

struct rpmsg_device *openamp_platform_get_rpmsg_vdev(int id);
```

这个是全志自己定义的 API，并非来自 OpenAMP 官方。全志平台的 OpenAMP 在初始化过程中会创建 RPMsg device，该 API 就是用来获取前面已经初始化好的 RPMsg device。一般来说，一个 RPMsg device 对应一个可进行通信的远端处理器。它具体的 ID 号，与初始化时传入给 openamp_platform_create_rpmsg_vdev 的 vdev_id 是对应的。

4.2.2 创建 endpoint

```
/* <openamp/rpmsg.h> */

/**
 * rpmsg_create_ept - create rpmsg endpoint and register it to rpmsg device
 *
 * Create a RPMsg endpoint, initialize it  with a name, source address,
 * remoteproc address, endpoint callback, and destroy endpoint callback,
 * and register it  to the RPMsg device.
 *
 * @ept: pointer to rpmsg endpoint
 * @name: service name associated to the endpoint
 * @src: local address of the endpoint
 * @dest: target address of the endpoint
 * @cb: endpoint callback
 * @ns_unbind_cb: end point service unbind callback, called when remote ept is
 *                destroyed.
```

```
 * In essence, an rpmsg endpoint represents a listener on the rpmsg bus, as
 * it binds an rpmsg address with an rx callback handler.
 *
 * Rpmsg client should create an endpoint to discuss with remote. rpmsg client
 * provide at least a channel name, a callback for message notification and by
 * default endpoint source address should be set to RPMSG_ADDR_ANY.
 *
 * As an option Some rpmsg clients can specify an endpoint with a specific
 * source address.
 */

int rpmsg_create_ept(struct rpmsg_endpoint *ept, struct rpmsg_device *rdev,
                     const char *name, uint32_t src, uint32_t dest,
                     rpmsg_ept_cb cb, rpmsg_ns_unbind_cb ns_unbind_cb);
```

对于同一个 RPMsg device，可以创建多个地址不同的 endpoint，提供给不同模块使用。

endpoint 的 cb 函数在远端处理器往该 endpoint 的地址发送一般的消息时会被调用到；而 ns_unbind_cb 函数会在远端对应的 endpoint 销毁时被调用到（因为要实现通信，远端会有一个对应的 endpoint 与本地的 endpoint 关联起来），如果不需要，可设为 NULL。

endpoint 的地址有一些特殊值：

- **0x35**（十进制的 53）：预留出来用作 name service 消息包的收发。
- **≥ 0x400**（十进制的 1024）：大于或等于 0x400 的值会被用作自动分配的地址值。当用户创建 endpoint 时，如果将 src 或 dest 设为 RPMSG_ADDR_ANY，RPMsg 会自动为其分配空闲的地址，这个分配的值就会从 0x400 开始。
- **RPMSG_ADDR_ANY**（十六进制的 0xFFFFFFFF）：在自动分配地址和 name service 通知机制中使用（使用时最好用 RPMsg 提供的宏 RPMSG_ADDR_ANY，以免该值后续发生改变）。

name service 机制，用于通知远端进行 RPMsg device/channel/endpoint 的动态申请，详细请见《DSP_OpenAMP_ 设计文档》。

当创建 endpoint 时，如果将 dest 设为 RPMSG_ADDR_ANY，会往远端发送 name service 消息包；等远端回应后，dest 就会变成远端对应 endpoint 的地址，此时就可开始通过本地的 endpoint 与远端的 endpoint 进行通信。

出于代码通用性的考虑，一般来说都会推荐将地址设为 RPMSG_ADDR_ANY，让其自动分配。

4.2.3 等待 endpoint 准备好

```
/* <openamp/rpmsg.h> */

/**
 * is_rpmsg_ept_ready - check if the rpmsg endpoint ready to send
 *
 * @ept: pointer to rpmsg endpoint
 *
 * Returns 1 if the rpmsg endpoint has both local addr and destination
```

```
 * addr set, 0 otherwise
 */
static inline unsigned int is_rpmsg_ept_ready(struct rpmsg_endpoint *ept)
{
    return ept && ept->rdev && ept->dest_addr != RPMSG_ADDR_ANY;
}
```

在第一次使用该 endpoint 向远端发送消息前，需要调用该 API 等待 endpoint 准备好（当使用 name service 机制时，等待实际就是远端是否已经进行了回应，是否已经修改了 endpoint 的 dest_addr）。

4.2.4 向远端发送数据

向远端发送数据可使用以下 API（具体的描述可参考头文件 openamp/rpmsg.h）：

```
/* <openamp/rpmsg.h> */

static inline int rpmsg_send(struct rpmsg_endpoint *ept, const void *data,
                int len);
static inline int rpmsg_sendto(struct rpmsg_endpoint *ept, const void *data,
                int len, uint32_t dst);
static inline int rpmsg_send_offchannel(struct rpmsg_endpoint *ept,
                uint32_t src, uint32_t dst,
                const void *data, int len);

static inline int rpmsg_trysend(struct rpmsg_endpoint *ept, const void *data,
                int len);
static inline int rpmsg_trysendto(struct rpmsg_endpoint *ept, const void *data,
                int len, uint32_t dst);
static inline int rpmsg_trysend_offchannel(struct rpmsg_endpoint *ept,
                uint32_t src, uint32_t dst,
                const void *data, int len);
```

- send 和 trysend 类别函数的区别主要是：当底层用来传输数据的 buffer 都满了时，send 类别的会阻塞住，直到 buffer 可用或 15 秒超时返回；trysend 类别的则是立即返回。
- send、sendto、send_offchannel 这三类函数的区别是：send 函数直接使用 ept 的 src 和 dst；sendto 函数使用 ept 的 src，但 dst 是使用额外传进来的参数；send_offchannel 函数则是 src 和 dst 都与 ept 无关，都是使用额外传进来参数。

由此可见，往另一端发送数据时，其实并不一定要用 endpoint 里的地址。如果之前创建有别的 endpoint，理论上已经可以往远端的任意地址发送数据。不过一般为了方便管理，还是更为推荐创建对应地址的 endpoint 来传输数据。

说明

从 OpenAMP 的内部代码实现来看，单次 rpmsg_send 发送的数据量无法超过底层 RPMsg 单个消息包 buffer 的大小，否则会被截断。当前 RPMsg 消息包 buffer 的大小固定是 512 bytes，扣除 16 bytes 的头部信息，用户单次可传输的数据量为 496 bytes。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.2.5 销毁 endpoint

```
/* <openamp/rpmsg.h> */

/**
 * rpmsg_destroy_ept - destroy rpmsg endpoint and unregister it from rpmsg
 *                     device
 *
 * @ept: pointer to the rpmsg endpoint
 *
 * It unregisters the rpmsg endpoint from the rpmsg device and calls the
 * destroy endpoint callback if it is provided.
 */
void rpmsg_destroy_ept(struct rpmsg_endpoint *ept);
```

4.3 Demo 程序

4.3.1 rpmsg 驱动层通信程序

DSP FreeRTOS SDK 中有一个 OpenAMP RPMsg 的 demo 程序，其源码路径为：components/thirdparty/rpmsg_demo。此 demo 的功能是创建一个通信端点，与 Linux 驱动进行通信，也可以实现 DSP 之间的通信。如果要实现与 Linux 应用层通信，可以参考rpmsg 应用层通信程序（rpmsg_test）章节。

4.3.1.1 编译

需要在 menuconfig 中选上：

```
Components options --->
  Thirdparty components --->
    [*] RPMsg demo        # CONFIG_COMPONENTS_RPMSG_DEMO
```

重新编译烧录固件后，在控制台中执行 help 命令可看到增加了 rpmsg_demo 命令。

4.3.1.2 基本使用

该 demo 的 help 信息如下：

```
USAGE:
 rpmsg_demo [OPTIONS]
OPTIONS:
 -c        : create endpoint
 -s MESSAGE  : send MESSAGE to remote
 -d        : destory endpoint
 -l        : list created endpoints
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
 -I ID      : specify rpmsg device ID (default: 0)
 -N NAME    : specify endpoint service name (default: "rpmsg_demo")
 -S SRC_ADDR : specify source address of endpoint (hexadecimal) (default is RPMSG_ADDR_ANY: 0xffffffff)
 -D DST_ADDR : specify destination address of endpoint (hexadecimal) (default is RPMSG_ADDR_ANY: 0xffffffff)

e.g.
 First, create an endpoint (its name should match that of remote rpmsg channel):
    rpmsg_demo -N "xxx" -c
 Then if remote sends message to it, the endpoint callback will be called.

 We can send message (e.g. "hello") to remote:
    rpmsg_demo -N "xxx" -s "hello"

 If this endpoint is no longer in use, destroy it:
    rpmsg_demo -N "xxx" -d
```

首先，使用 -c 选项创建 endpoint（可使用 -N、-S、-D 指定名字和地址）。通过 -l 选项可列出之前创建过的 endpoint。

创建了 endpoint 之后，如果远端有消息发送过来，其回调函数会被调用到，将收到的数据打印出来。

使用 -s 选项可将某个字符串发送到远端。

如果不再使用该 endpoint，使用 -d 参数进行销毁。

技巧

RPMsg 的总线将会寻找符合-N后指定名字的 RPMsg 驱动，因此下面的例子将会与 Linux 内核的 RPMsg 示例驱动代码 drivers/rpmsg/sunxi_rpmsg_client_sample.c 配合起来使用。

4.3.1.3 简单例子

以下是一个使用 rpmsg_demo 的简单例子：

```
Linux与DSP0/1通信：

（ID默认是0，NAME默认rpmsg_demo）

1、DSP0/1输入rpmsg_demo -N rpmsg-client-sample -c，Linux端会加载sunxi_rpmsg_client_sample.c的驱动代码，自动
      创建对应通信用的endpoint

2、DSP0/1输入rpmsg_demo -N rpmsg-client-sample -s "123"，Linux端会收到消息并自动发回DSP0/1

3、DSP0/1输入rpmsg_demo -N rpmsg-client-sample -d，销毁endpoint
```

```
DSP0与DSP1通信：

（ID默认是0，NAME默认rpmsg_demo）

1、DSP0输入rpmsg_demo -l 1 -c

2、DSP1输入rpmsg_demo -l 1 -c

3、DSP0输入rpmsg_demo -l 1 -s "123"，DSP1端会收到消息并自动发回DSP0
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
4、DSP1输入rpmsg_demo -I 1 -s "456"，DSP0端会收到消息并自动发回DSP1

5、DSP0/1输入rpmsg_demo  -I 1 -d，销毁endpoint
```

4.3.2 rpmsg 应用层通信程序（rpmsg_test）

此程序只适用于 DSP 和 Linux 的通信，对于 DSP 之间的通信，可以参考rpmsg 驱动层通信程序

DSP 程序：rpmsg_test，源码路径：components/thirdparty/openamp/rpmsg_ctrl/test

Linux 配合使用的程序：rpmsg_test，源码路径：package/allwinner/rpmsg/test

4.3.2.1 编译

DSP 编译时需要在 menuconfig 中选上：

```
Components options --->
  Thirdparty components --->
    OpenAMP components --->
      [*] rpmsg ctrl driver
      [*]   rpmsg ctrl test cmds
```

Linux 编译时需要在 menuconfig 中选上：

```
Allwinner  --->
  System  --->
    -*- librpmsg..................................... rpmsg library
    <*> rpmsg_test.................................... rpmsg test demo
```

4.3.2.2 基本使用

对于 DSP 而言，该 demo 的 help 信息如下：

```
USAGE:
 rpmsg_test [OPTIONS]
OPTIONS:
 -h       : print help message
 -c       : create endpoint
 -d       : destory endpoint
 -v       : verbosely print check result
 -M        : max rpmsg client (default: 10)
 -t tx_delay : specify tx_delay_ms (Global variable, default: 500)
 -L tx_len   : specify tx length (Global variable, default: 32 bytes)

e.g.
    rpmsg_test -N "xxx" -L LENGTH -c
    rpmsg_test -N "xxx" -L LENGTH -d
```

对于 Linux 而言，该 demo 的 help 信息如下：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
USAGE:
  rpmsg_test [OPTIONS]

OPTIONS:
  -r name    : remoteproc process name.
  -c name    : create specifies name endpoint
  -L tx_len  : specifies tx_len
  -t ms      : specifies tx_delay
  -v         : printf verbose info
  -R threshold : rx warn threshold
  -T threshold : tx warn threshold
e.g.
  rpmsg_test -r dsp_rproc@0 -c test -L 64 -t 500: create an endpoint to communicate with dsp_rproc@0 called test
```

4.3.2.3 简单例子

以下是一个使用 rpmsg_test 的简单例子：

先确保 Linux 有以下两个 rpmsg 的控制节点，如果没有，请检查 DSP 是否有正常启动。

```
root@TinaLinux:/# ls -l /dev/rpmsg*
crw-------   1 root     root      253,   0 Jan  3 04:45 /dev/rpmsg_ctrl-dsp0_rproc@0
crw-------   1 root     root      253,   1 Jan  3 04:46 /dev/rpmsg_ctrl-dsp1_rproc@0
```

```
Linux与DSP0通信：

1、DSP0输入rpmsg_test -N cpu_dsp0 -c -v

2、Linux输入rpmsg_test -r dsp0_rproc@0 -c cpu_dsp0 -v
```

```
Linux与DSP1通信：

1、DSP1输入rpmsg_test -N cpu_dsp1 -c -v

2、Linux输入rpmsg_test -r dsp1_rproc@0 -c cpu_dsp1 -v
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5 使用 RPBuf 进行核间通信

5.1 RPBuf 介绍

RPBuf（Remote Processor Buffering）是为了能在异构的多处理器之间高效传输大量数据（单次大于 496byte）而设计出来。Linux 原生的 RPMsg 框架已经可以支持在异构核间进行传输数据，但 RPMsg 单次传输的长度最大为 512 bytes（在 Linux 内核代码中用宏来定义），除去头部的 16 bytes，有效数据最大为 496 bytes，除此之外，被传输的数据本身需要被拷贝到共享内存，远端处理器收到通知后再从共享内存中将数据拷贝出来，整个传输过程不可避免地存在 “写入” 和 “读出” 两次拷贝。因此若要传输大量数据，会存在多次拷贝和频繁的核间中断，并不高效。在全志平台中，针对单次传输数据量大于 496bytes 的情况，我们额外设计了 RPBuf 框架。

RPBuf 同样依赖于共享内存来传输数据，但两端的处理器都能直接通过地址访问共享内存，当把数据拷贝到共享内存后，会将数据所在位置的地址偏移和长度使用 rpmsg 告知另一端，让另一端自行从该地址读取数据，从而避免额外的拷贝，实现单次也能传输大量数据。

5.2 RPBuf 传输速率

5.2.1 Linux 收 DSP 发

以下结果均是发送一百次，取平均值

场景 1：使用rpbuf_test收发，两处理器基于外部定时器作时间同步，来进行测试

大小	异步传输（刷 cache 耗时）	同步传输（刷 cache 耗时）
4K	13 us（0.7 us）	665 us（0.7 us）
256K	13 us（14 us）	628 us（14 us）
512K	13 us（28 us）	532 us（28 us）
1M	13 us（55 us）	592 us（55 us）

场景 2：基于上面的测试，DSP 额外在运行memtester，再进行测试

大小	异步传输（刷 cache 耗时）	同步传输（刷 cache 耗时）
4K	17 us（0.7 us）	712 us（0.7 us）
256K	38 us（34 us）	588 us（24 us）

版权所有 © 珠海全志科技股份有限公司。保留一切权利

大小	异步传输（刷 cache 耗时）	同步传输（刷 cache 耗时）
512K	38 us（108 us）	669 us（88 us）
1M	18 us（85 us）	567 us（75 us）

技巧

从上述结果看，数据的大小并不直接影响传输的速率，但会影响刷的 cache 耗时；从异步传输的结果来看，发送端的系统负载会影响传输速率；从同步传输的结果来看，发送端与接收端实际运行环境的系统负载，任务数量，中断频次等均会影响传输速率。

5.3 RPBuf 相关概念与流程

5.3.1 RPBuf 框架基本组件

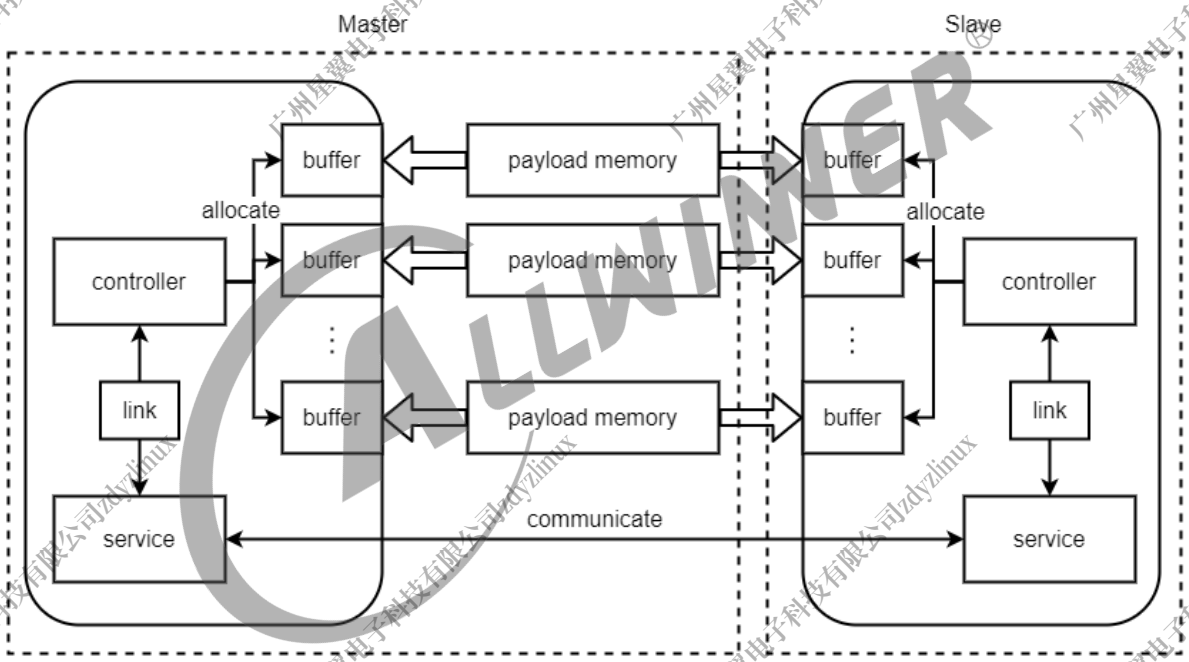

图 5-1: RPBuf Components

- **controller**：供用户操作用来申请 buffer 的接口，一个 controller 对应一个远端处理器。
- **service**：在 RPBuf 内部负责与远端处理器通信的组件，使用的是 RPmsg 机制。
- **link**：负责将 controller 和 service 连接起来。每一个 controller 都必须与一个 service 连接起来，否则它无法与远端通信。
- **buffer**：表示申请到的共享内存。用户通过操作 buffer 对象，可直接访问对应的共享内存。对于同一片共享内存，在通信的两端都各有一个对应的 buffer 对象，它们的名字和长度相同。
- **payload memory**：用来存放实际传输数据的共享内存，因此称为 payload（有效负载）。它统一由 master 端分配，在两端都各有名字相同的 buffer 对象与其对应。

使用 RPBuf 通信的两端处理器必然有一端作为 master，另一端作为 slave。master 和 slave 对

于 buffer 均可进行读写，并没有方向上的限制。两者唯一的区别在于 payload memory 是统一由 master 分配，slave 只负责使用。

将 RPBuf 内部分为 controller 和 service 两大组件，如此设计的目的在于将 “buffer 管理” 和 “通信” 两个功能解耦。对于内存管理方式不同的平台，可以定义不同的 controller；对于通信方式不同的平台，也可以定义不同的 service，从而做到将不同的 controller 实现和不同的 service 实现进行自由组合。

link 通过名为 **token** 的结构判断哪个 controller 跟哪个 service 连接。每一个 controller 和 service 在注册时都需要指定一个 token，只有 token 相同的才能连接起来。

5.3.2 申请/释放 buffer

用户使用 RPBuf 申请共享 buffer 时，需要在 master 端和 slave 端都指定相同的 “名字” 和 “长度”，否则两端无法匹配上，并且只有在两端都申请成功后，buffer 才可被使用（该状态称为 “**available**”）。

说明

buffer 的 payload memory 是只由 master “分配”，但 “申请 buffer” 这一操作，master 和 slave 都要自己主动进行。

对于 master 和 slave 孰先孰后申请 buffer，并没有什么要求，只要能确保两端最后都申请了就行。在 controller 内部会维护以下三个结构，用来区分 buffer 当前处于什么状态：

- remote_dummy_buffers：用来记录远端已申请、本地未申请的 buffer。
- local_dummy_buffers：用来记录本地已申请、远端未申请的 buffer。
- buffers：用来记录本地和远端均已申请的 buffer。只有在这里的 buffer 才会被认为是 “available”。

申请 buffer 时其状态切换如下所示：

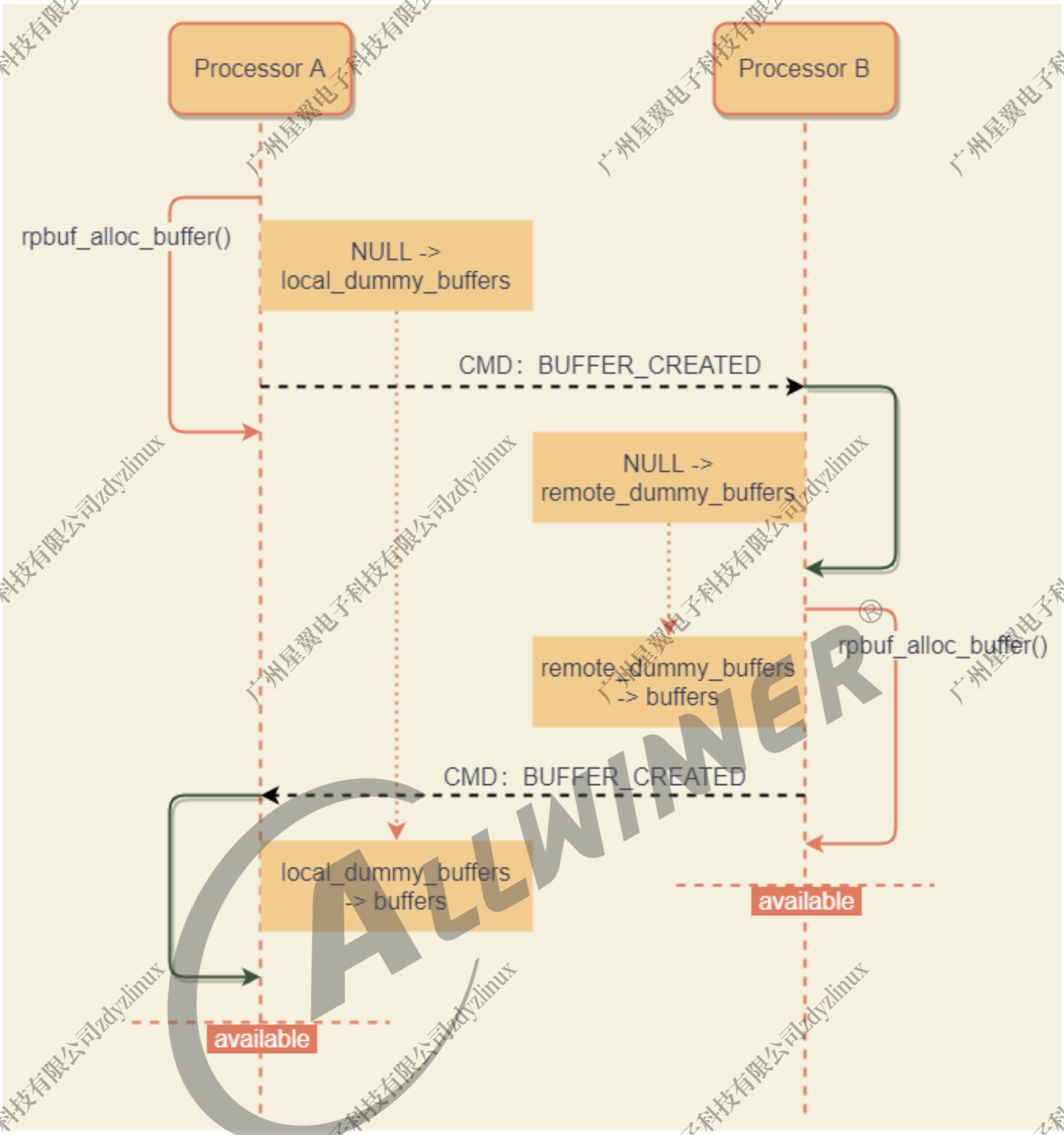

图 5-2: RPBuf Allocate Buffer Flowchart

释放 buffer 时也类似，无需区分先后顺序：

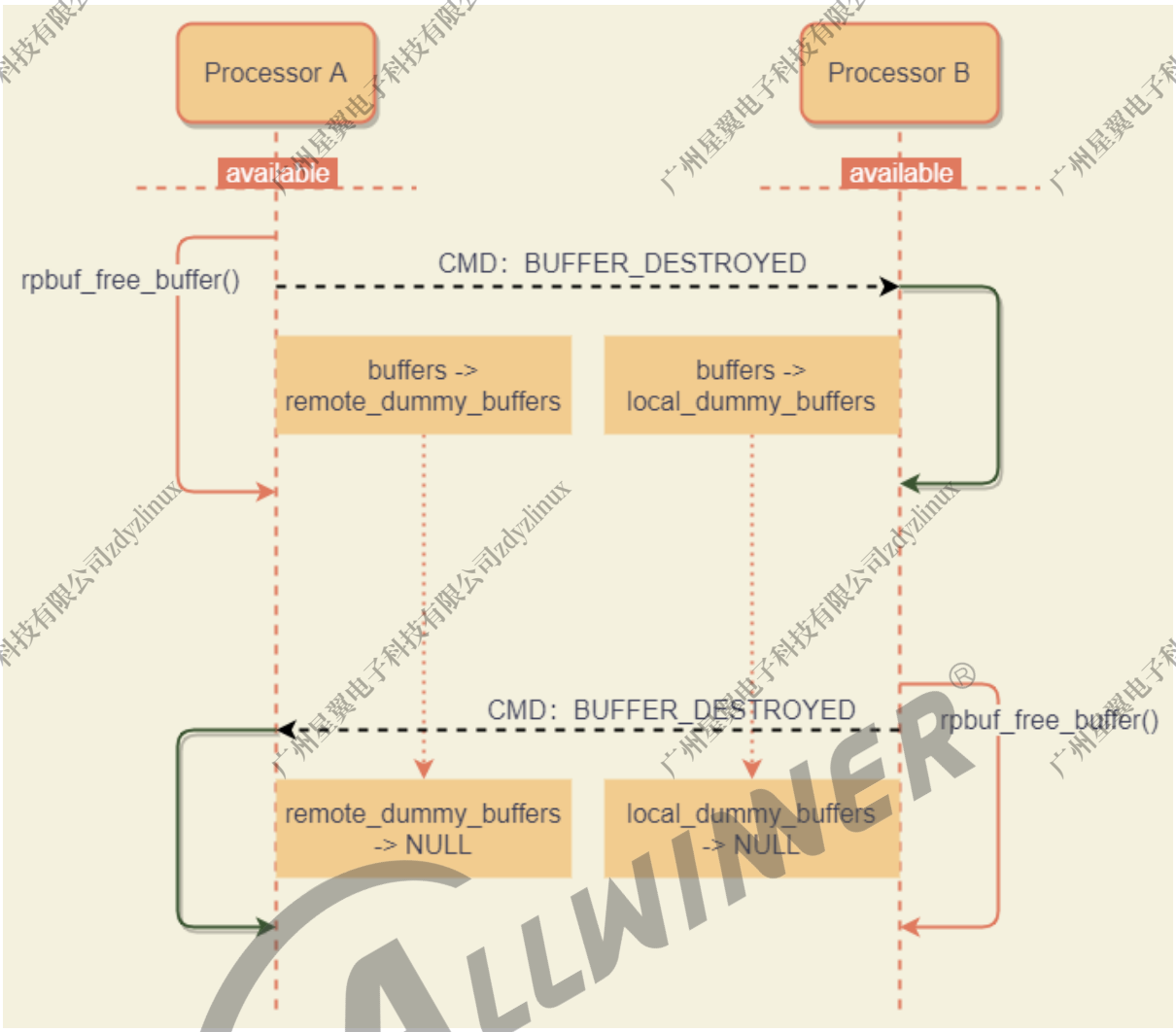

图 5-3: RPBuf Free Buffer Flowchart

⚠注意

在任意一端释放了 buffer 后，实际对两端来说该 buffer 都已经处于 ``not available'' 的状态，此时都不应该对 buffer 进行任何读写操作。但因为 buffer 的地址是直接暴露给用户，RPBuf 框架无法限制用户对该地址的直接访问，因此用户需自行确保不会在 not available 时读写 buffer。

上面流程图描述的是正常情况下 buffer 的状态切换，而在异常情况下（如通信失败、远端处理器崩溃后重新运行等），需要进行一些特殊处理。下图描述的是从单个处理器自身的角度来看，当它的 buffer 分别处于 NULL、local_dummy_buffers、remote_dummy_buffers、buffers 时，在本地执行 rpbuf_alloc_buffer()/rpbuf_free_buffer() 或收到远端传过来的 RPBUF_SERVICE_CMD_BUFFER_CREATED /RPBUF_SERVICE_CMD_BUFFER_DESTROYED 后，会如何处理：

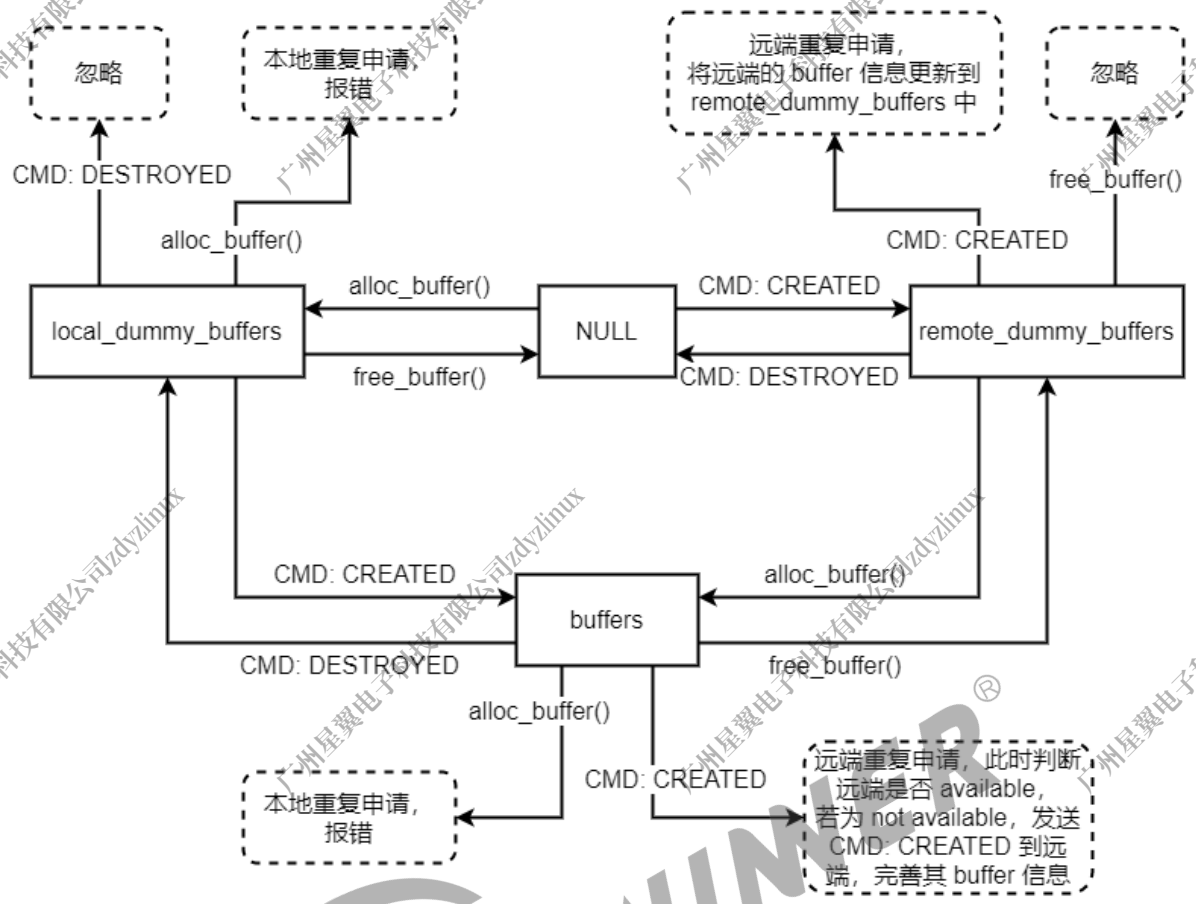

图 5-4: RPBuf Buffer States

广州星翼电子科技有限公司zdyzlinux

5.3.3 数据传输流程

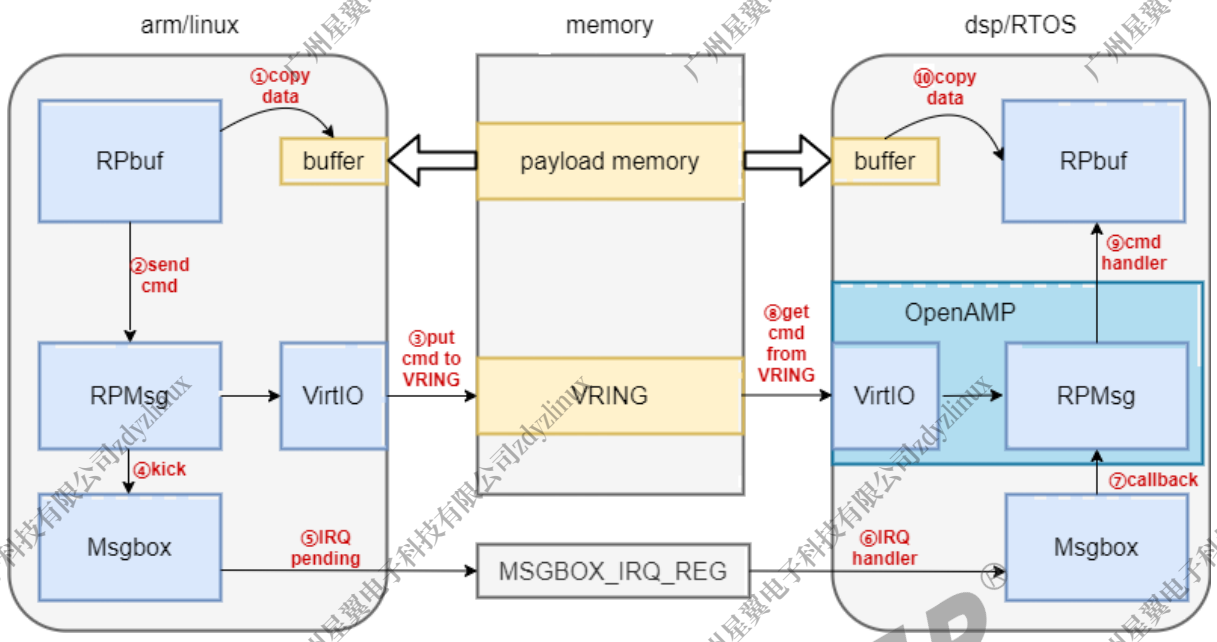

图 5-5: RPBuf Wrok

- **RPBuf**：基于共享内存和 RPMsg 消息通知，实现传输大数据传输的框架。
- **RPMsg**：基于 VirtIO 管理的共享内存，实现数据传输的框架。
- **VirtIO**：原本是一套用在虚拟化环境中传输数据的框架，这里用作共享内存 (VRING) 的管理。
- **OpenAMP**：OpenAMP 框架为 RTOS、裸机和 Linux 用户空间提供了 RPMsg、VirtIO、remoteproc（未列出）的实现，并且与 Linux 内核兼容。
- **Msgbox**：是全志平台提供的一套消息中断机制，已通过 linux 内核中原生的 mailbox 框架作适配。
- **MSGBOX_IRQ_REG**：Msgbox 的中断相关寄存器。
- **buffer**：表示申请到的共享内存。用户通过操作 buffer 对象，可直接访问对应的共享内存。
- **payload memory**：用来存放实际传输数据的共享内存，因此称为 payload（有效负载）。
- **VRING**：由 VirtIO 管理的一个环形共享内存。

上图展示了一次 RPBuf 数据传输的通信过程，下面详细说明：

1. arm 端把数据拷贝到 buffer 中，在初始化时已经将 buffer 和 payload memory 地址绑定，因此数据拷贝后相当于存放到了 payload memory 中。
2. 在消息传输命令后加上数据在 payload memory 中的起始地址和长度，组成数据包，调用 RPMsg 接口发送，在驱动中我们定义了以下几个命令。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

cmd	功能
RPBUF_SERVICE_CMD_BUFFER_CREATED	创建命令，通知另一端处理器创建RPBuf。
RPBUF_SERVICE_CMD_BUFFER_DESTROYED	释放命令，通知另一端处理器释放RPBuf。
RPBUF_SERVICE_CMD_BUFFER_TRANSMITTED	消息传输命令，传输数据的起始地址和长度给另一端处理器。
RPBUF_SERVICE_CMD_BUFFER_ACK	消息同步命令，用于收发消息的同步。

1. RPMsg 通过调用 VirtIO 的接口，把数据包存放到 VRING 中。
2. RPMsg 通过 kick 的机制，使 Msgbox 产生一个中断。
3. Msgbox 的中断标记位置 1。
4. dsp 端的 Mgxbox 接收到中断后，触发其中断处理函数，并把数据放到队列中。
5. Mgxbox 驱动会有一个 task 不断等待队列的数据，当有数据则取出并调用 RPMsg 初始化时注册的 callback。
6. 在 callback 中会调用 VirtIO 的接口，把数据包从 VRING 中取出。
7. 调用 RPBuf 的消息处理函数，解析数据包，根据不同的 cmd 执行不同的 handler，这里收到的 cmd 是 RPBUF_SERVICE_CMD_BUFFER_TRANSMITTED。
8. 根据接收到的地址和长度从 payload memory 中拷贝出数据。

说明

在每次数据拷贝之后，都需要使用 RPMsg 发送消息传输命令，对于单次传输字节数小于 496bytes 时，使用 RPBuf 反而会降低性能，因为不仅需要用 RPMsg 发消息，还多了两次从 payload memory 中拷贝数据的操作，因此建议数据量较小时使用 RPMsg。

5.3.4 数据同步

当本地处理器将数据拷贝至 payload memory 并发送消息传输命令到远端处理器后，并不知道远端处理器在何时会取走消息，假如远端处理器没有及时取走数据，那么本地处理器可能会再次拷贝造成数据覆盖，旧数据丢失的问题，因此 RPBbuf 在驱动中增加一个数据同步的机制。

5.3.4.1 在 RTOS 中设置同步

```
int rpbuf_buffer_set_sync(struct rpbuf_buffer *buffer, bool sync)
```

5.3.4.2 在 linux 用户空间中设置同步

```
int rpbuf_set_sync_buffer(rpbuf_buffer_t *buffer, uint32_t is_sync)
```

5.3.4.3 数据同步流程

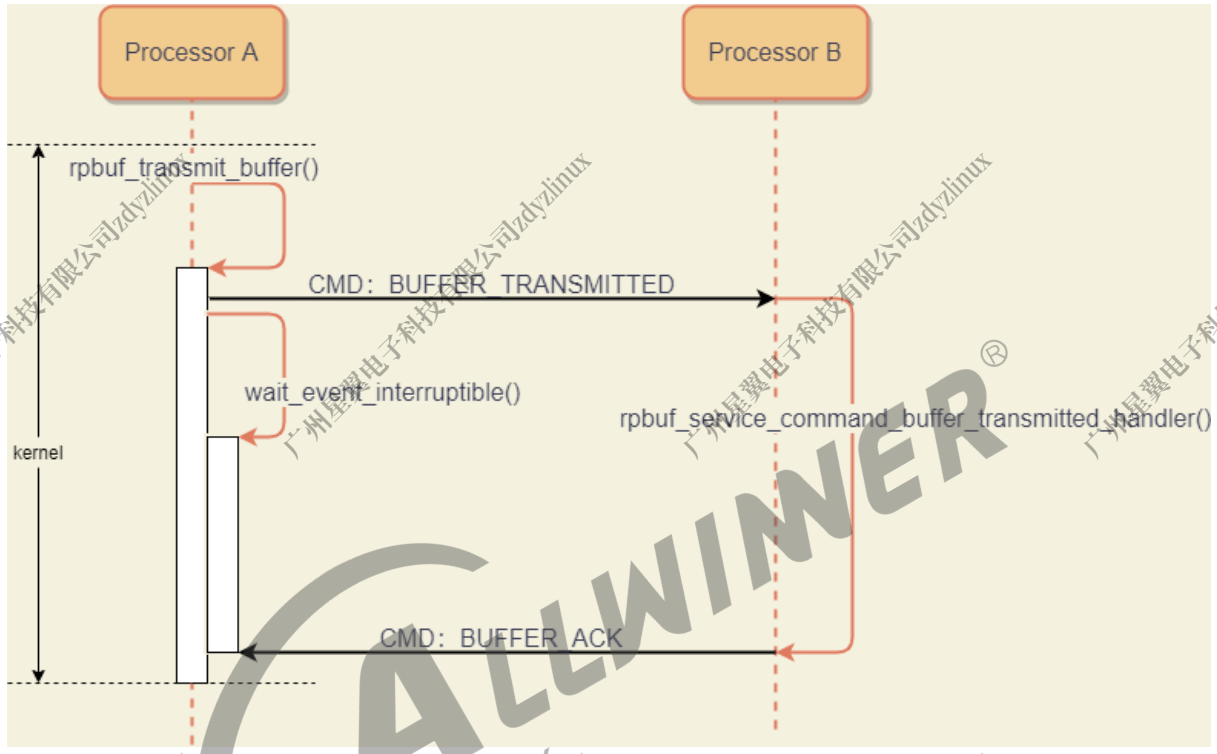

图 5-6: RPBuf Ack Flowchart

linux 应用层先把数据拷贝到共享内存中，然后调用 rpbuf 的传输接口，这会从应用层切换到内核中，调用rpbuf_transmit_buffer()，这会发送一个消息传输命令RPBUF_SERVICE_CMD_BUFFER_TRANSMITTED，然后在wait_event_interruptible()中阻塞，DSP 收到消息后会根据消息的 CMD 调用不同的处理函数，当解析到消息传输命令会调用rpbuf_service_command_buffer_transmitted_handler()，在里面会根据收到的偏移地址和长度刷 cache，然后调用应用中注册的 rx_cb，最后发送一个消息同步命令RPBUF_SERVICE_CMD_BUFFER_ACK，linux 收到消息同步命令之后会调用rpbuf_service_command_buffer_ack_handler()，在里面会唤醒等待事件，然后内核的阻塞线程被唤醒，内核切换到应用层中继续执行往下的逻辑。

5.3.5 异步通知

在应用层中调用rpbuf_install_rx_signal()，可以让内核使用异步通知，当内核收到消息传输命令后，会发送一个SIGIO信号给应用层，然后应用层捕获该信号后会调用其处理函数，处理函数通过第二个参数sighandler_t handler进行安装，具体用法可以参考rpbuf_demo 程序章节里 linux 的源码。

说明

目前 linux 每个进程只支持一个 rpbuf 使用异步通知机制，因为 rpbuf 接收消息后内核发出的信号是 SIGIO，进程捕获信号后只会执行对应处理函数，并不会关注该信号从何处来的，也不知道到底是哪一个 rpbuf 收到的消息发出了信号，假如有多个 rpbuf 都调用了rpbuf_install_rx_signal()，那么只会触发最后安装的处理函数。

5.4 代码分析

5.4.1 Linux

Linux 内核的 RPBuf 代码如下：

```
.
├── drivers
│   └── rpbuf
│       ├── rpbuf_controller_sunxi.c   // 全志平台的 rpbuf controller 实现
│       ├── rpbuf_core.c               // rpbuf 核心代码
│       ├── rpbuf_dev.c                // 在 /dev 下提供节点供用户空间操作
│       ├── rpbuf_internal.h           // rpbuf 内部头文件
│       ├── rpbuf_sample_sunxi.c       // 在内核使用 rpbuf 的一个例子，也提供一些 sysfs 节点供简单调试使用
│       └── rpbuf_service_rpmsg.c      // 通过 rpmsg 实现的 rpbuf service
└── include
    └── linux
        └── rpbuf.h                    // 提供给其他驱动使用的 rpbuf 头文件
```

Tina 系统的 package 中提供了在用户空间使用 rpbuf 的库和示例 demo：

```
.
└── package
    └── allwinner
        └── rpbuf
            ├── demo/        // 用户空间使用 rpbuf 的示例 demo，依赖于 librpbuf
            ├── test/        // 用户空间压测 rpbuf 的示例 demo，依赖于 librpbuf
            └── librpbuf/    // 将与内核 rpbuf_dev 交互的操作封装成接口供其他应用程序使用
```

5.4.1.1 在内核空间使用 RPBuf

供其他内核模块使用的 RPBuf 接口都声明在头文件 include/linux/rpbuf.h 中，在使用上大致分为以下步骤：

1. 创建并使能 RPBuf controller（后面以 rpbuf_controller_sunxi 为例来说明）。
2. 创建并使能 RPBuf service（后面以 rpbuf_service_rpmsg 为例来说明）。
3. 获取到 controller 的指针（一个 controller 对应一个远端处理器）。
4. 通过 controller 申请 buffer（rpbuf_alloc_buffer()），待 buffer available 后可对其地址直接进行读写。
5. 使用 rpbuf_transmit_buffer() 将数据传输到远端；若远端有数据传输过来，该 buffer 的 rx_cb() 会被触发。
6. 使用结束后，释放 buffer（rpbuf_free_buffer()）。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5.4.1.1.1 创建并使能 rpbuf_controller_sunxi

除了要使能编译 rpbuf_controller_sunxi.c 之外，我们还需要在 DTS 中定义对应的 rpbuf_controller device，以下是一个例子：

```
dsp0_rpbuf_controller: rpbuf_controller@0 {
  compatible = "allwinner,rpbuf-controller";
  remoteproc = <&dsp0_rproc>;
  ctrl_id = <0>;   /* index of /dev/rpbuf_ctrl */
  memory-region = <&dsp0_rpbuf_reserved>;  /* optional */
  status = "okay";
};
```

- remoteproc 表示它对应哪个远端处理器，它会以该 remoteproc device 作为 token。
- ctrl_id 表示它生成的供用户空间操作的节点 /dev/rpbuf_ctrlX 的编号（即这里的 X 值）。
- memory-region（可选）用于将一片预留内存赋给 controller，让其从中申请 payload memory。例如若我们定义了如下的 reserved-memory dsp0_rpbuf_reserved，则 dsp0_rpbuf_controller 默认都会从首地址为 0x42244000、长度为 0x2000 这片内存区域来申请 payload memory。

```
reserved-memory {
  #address-cells = <2>;
  #size-cells = <2>;
  ranges;

  dsp0_rpbuf_reserved:dsp0_rpbuf@42244000 {
    compatible = "shared-dma-pool";
    no-map;
    reg = <0x0 0x42244000 0x0 0x2000>;
  };
};
```

当 rpbuf_controller_sunxi 驱动加载时，会调用 rpbuf_create_controller() 和 rpbuf_register_controller() 来创建和注册 controller，此时需要指定 enum rpbuf_role，表示该 RPBuf 是作为 master 还是 slave。

```
/* "rpbuf_internal.h" */

struct rpbuf_controller *rpbuf_create_controller(struct device *dev,
                                const struct rpbuf_controller_ops *ops,
                                void *priv);
void rpbuf_destroy_controller(struct rpbuf_controller *controller);

int rpbuf_register_controller(struct rpbuf_controller *controller,
                    void *token, enum rpbuf_role role);
void rpbuf_unregister_controller(struct rpbuf_controller *controller);
```

在调用 rpbuf_create_controller() 时，我们还可以自定义一些 ops，来覆盖 RPBuf 默认的行为。详见章节自定义 ops。

5.4.1.1.2 创建并使能 rpbuf_service_rpmsg

rpbuf_service_rpmsg 是依赖 RPMsg 框架实现，它本质上是一个 rpmsg_driver，因此它的初始化是靠 RPMsg 的 name service 机制实现：只有远端的 rpbuf_service_rpmsg 在初始化过程中给 Linux 发送 name service 消息包后，才会触发 linux 加载这端的 rpbuf_service_rpmsg。这整个过程对 RPBuf 用户来说是隐式的，无法直接感知，但如果 RPBuf service 未初始化完成，调用相关 API 操作 buffer 时会报错，因为 RPBuf 是当且仅当 controller 和 service 都初始化完成后才能正常使用。

当 rpbuf_service_rpmsg 驱动加载时，类似于 controller，也会调用 rpbuf_create_service() 和 rpbuf_register_service () 创建和注册 service。

```
/* "rpbuf_internal.h" */

struct rpbuf_service *rpbuf_create_service(struct device *dev,
                                           const struct rpbuf_service_ops *ops,
                                           void *priv);
void rpbuf_destroy_service(struct rpbuf_service *service);

int rpbuf_register_service(struct rpbuf_service *service, void *token);
void rpbuf_unregister_service(struct rpbuf_service *service);
```

Linux 的 rpbuf_service_rpmsg 驱动会从它的父设备中寻找实际使用的 remoteproc device，以它作为 token。因为不同内核版本的 remoteproc/RPMsg 代码有异，寻找 remoteproc device 的方式也会有所差异：

Linux-4.9 是往上第 3 个父设备：

```
    /*
     * We should find the underlying remoteproc device.
     *
     * Normally, the rpmsg device is created from:
     *      remoteproc device   <---  defined by vendors, usually in device tree
     *      |---  rproc         <---  rproc_alloc()
     *            |---  virtio        <---  rproc_handle_vdev() & register_virtio_device()
     *                  |---  rpmsg   <---  rpmsg_register_device()
     *
     * Therefore the 3rd parent of rpmsg device is the underlying remoteproc
     * device.
     */
    dev_tmp = dev;
    for (i = 0; i < 3; i++) {
        dev_tmp = dev_tmp->parent;
        if (! dev_tmp) {
            dev_err(dev, "cannot get rpmsg device parent %d\n", i);
            ret = -ENOENT;
            goto err_destroy_service;
        }
        dev_dbg(dev, "rpmsg device parent %d: %s\n", i, dev_name(dev_tmp));
    }

    /* We use the underlying remoteproc device as rpbuf link token */
    ret = rpbuf_register_service(service, dev_tmp);
    if (ret < 0) {
        dev_err(dev, "rpbuf_register_service failed\n");
```

```
        goto err_destroy_service;
    }
```

Linux-5.4 是往上第 4 个父设备：

```
    /*
     * We should find the underlying remoteproc device.
     *
     * Normally, the rpmsg device is created from:
     *      remoteproc device   <---  defined by vendors, usually in device tree
     *      |---  rproc       <---  rproc_alloc()
     *            |---  rproc_vdev    <----  rproc_handle_vdev()
     *                  |---  virtio       <---  register_virtio_device()
     *                        |---  rpmsg    <---  rpmsg_register_device()
     *
     * Therefore the 4th parent of rpmsg device is the underlying remoteproc
     * device.
     */
    dev_tmp = dev;
    for (i = 0; i < 4; i++) {
        dev_tmp = dev_tmp->parent;
        if (! dev_tmp) {
            dev_err(dev, "cannot get rpmsg device parent %d\n", i);
            ret = -ENOENT;
            goto err_destroy_service;
        }
        dev_dbg(dev, "rpmsg device parent %d: %s\n", i, dev_name(dev_tmp));
    }

    /* We use the underlying remoteproc device as rpbuf link token */
    ret = rpbuf_register_service(service, dev_tmp);
    if (ret < 0) {
        dev_err(dev, "rpbuf_register_service failed\n");
        goto err_destroy_service;
    }
```

5.4.1.1.3 获取 controller 的指针

若某一个模块的驱动想要使用 RPBuf，它需要先获取到对应 controller 的指针，方法如下：

首先在该驱动模块的 DTS 中增加一个 rpbuf 节点，并将对应的 controller device 节点传给它，以 rpbuf_sample_sunxi.c：

```
rpbuf_sample: rpbuf_sample@0 {
    compatible = "allwinner,rpbuf-sample";
    rpbuf = <&dsp0_rpbuf_controller>;
    status = "okay";
};
```

如果需要跟多个远端通信，则传递多个 controller：

```
rpbuf_sample: rpbuf_sample@0 {
    compatible = "allwinner,rpbuf-sample";
    rpbuf = <&dsp0_rpbuf_controller>, <&dsp1_rpbuf_controller>;
    status = "okay";
};
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

然后通过 rpbuf_get_controller_by_of_node() 即可获取到对应的 controller 指针：

```
/* <linux/rpbuf.h> */

/**
 * rpbuf_get_controller_by_of_node - get rpbuf controller by device of_node
 * @np: of_node of the device
 * @index: index to select which of_node to use
 *
 * Return pointer to the rpbuf_controller on success, or NULL on failure.
 */
struct rpbuf_controller *rpbuf_get_controller_by_of_node(const struct device_node *np, int index);
```

5.4.1.1.4 申请/使用/释放 buffer

驱动操作 RPBuf buffer 的 API 位于头文件 include/linux/rpbuf.h 中，主要有如下：

```
typedef void (*rpbuf_available_cb_t)(struct rpbuf_buffer *buffer, void *priv);
typedef int (*rpbuf_rx_cb_t)(struct rpbuf_buffer *buffer, void *data, int data_len, void *priv);
typedef int (*rpbuf_destroyed_cb_t)(struct rpbuf_buffer *buffer, void *priv);

struct rpbuf_buffer_cbs {
    rpbuf_available_cb_t available_cb;
    rpbuf_rx_cb_t rx_cb;
    rpbuf_destroyed_cb_t destroyed_cb;
};

struct rpbuf_buffer *rpbuf_alloc_buffer(struct rpbuf_controller *controller,
                const char *name, int len,
                const struct rpbuf_buffer_ops *ops,
                const struct rpbuf_buffer_cbs *cbs,
                void *priv);
int rpbuf_free_buffer(struct rpbuf_buffer *buffer);

int rpbuf_buffer_is_available(struct rpbuf_buffer *buffer);

int rpbuf_transmit_buffer(struct rpbuf_buffer *buffer,
            unsigned int offset, unsigned int data_len);

const char *rpbuf_buffer_name(struct rpbuf_buffer *buffer);
int rpbuf_buffer_id(struct rpbuf_buffer *buffer);
void *rpbuf_buffer_va(struct rpbuf_buffer *buffer);
phys_addr_t rpbuf_buffer_pa(struct rpbuf_buffer *buffer);
u64 rpbuf_buffer_da(struct rpbuf_buffer *buffer);
int rpbuf_buffer_len(struct rpbuf_buffer *buffer);
void *rpbuf_buffer_priv(struct rpbuf_buffer *buffer);
```

- rpbuf_alloc_buffer() 用于通过 controller 申请 buffer。对同一个 buffer，本地与远端的 name 和 len 必须相同。此时注册的 rpbuf_buffer_cbs 回调函数，会在 buffer 的不同阶段被调用到：
- available_cb：当 buffer 变为 available 时会被触发。
- rx_cb：当远端有数据传输过来后会被触发，从 data 和 data_len 中可获取到数据的首地址和长度。
- destroyed_cb：当远端 buffer 被释放后会被触发。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- rpbuf_free_buffer() 用于释放 buffer（会触发远端的 destroyed_cb()）。
- rpbuf_is_available() 用于判断当前 buffer 是否为 available。一般情况下如果依靠 available_cb 和 destroyed_cb 来监视 buffer 的 available 状态了，就不需要再用 rpbuf_is_available() 来检查了。
- rpbuf_transmit_buffer() 用于将数据传输到远端，会触发远端的 rx_cb()。其中 offset 为数据所在的首地址相对于 buffer 首地址的偏移，data_len 是数据的长度。这允许每次传输时只传 buffer 的一部分，不必将整个 buffer 都传过去。
- 其他 API 用于获取 buffer 的一些信息（因为 struct rpbuf_buffer 的定义不会直接暴露给用户）。其中 rpbuf_buffer_va() 返回的是可供用户驱动代码直接访问的 buffer 首地址的虚拟地址（Virtual Address）。

⚠ 注意

1. **rpbuf_transmit_buffer() 时传入的 offset 是 ``数据首地址相对于 buffer 首地址的偏移''，而远端 rx_cb() 被触发后，其 data 指针指向的是 ``数据的首地址''，而非偏移，即：data = buffer 首地址 + offset，需注意区分。**
2. **因为 RPbuf 是在异构处理器之间传输数据，期间会涉及刷 cache 的操作，使用 rpbuf_transmit_buffer() 时需注意传输的数据是否按 cacheline 对齐（cacheline 大小视具体的处理器平台而定），否则可能导致无关区域的数据受影响。**

5.4.1.1.5 自定义 ops

在创建 controller 或申请 buffer 时，可通过一些自定义的 ops 来覆盖 RPBuf 默认的行为，其优先级为 “buffer > controller > 默认”。

```
/* <linux/rpbuf.h> */

/**
 * struct rpbuf_buffer_ops - user-specific rpbuf buffer operations
 * @alloc_payload_memory: allocate memory for payload
 * @free_payload_memory: free memory for payload
 * @addr_remap: remap pa or da to va
 * @buf_dev_mmap: mmap file operation for buf_dev
 */
struct rpbuf_buffer_ops {
    int (*alloc_payload_memory)(struct rpbuf_buffer *buffer, void *priv);
    void (*free_payload_memory)(struct rpbuf_buffer *buffer, void *priv);
    void *(*addr_remap)(struct rpbuf_buffer *buffer,
                phys_addr_t pa, u64 da, int len, void *priv);
    int (*buf_dev_mmap)(struct rpbuf_buffer *buffer, void *priv, struct vm_area_struct *vma);
};
```

```
/* "rpbuf_internal.h" */

struct rpbuf_controller_ops {
    int (*alloc_payload_memory)(struct rpbuf_controller *controller,
                struct rpbuf_buffer *buffer, void *priv);
    void (*free_payload_memory)(struct rpbuf_controller *controller,
                struct rpbuf_buffer *buffer, void *priv);
    void *(*addr_remap)(struct rpbuf_controller *controller,
                phys_addr_t pa, u64 da, int len, void *priv);
```

```
    int (*buf_dev_mmap)(struct rpbuf_controller *controller,
                struct rpbuf_buffer *buffer, void *priv,
                struct vm_area_struct *vma);
};
```

- alloc_payload_memory 与 free_payload_memory 只在 master 端使用，用于申请/释放 payload memory。
- addr_remap 只在 slave 端使用，用于将 master 端在申请 payload memory 后传过来的 pa（Physical Address）或 da（Device Address）转换为可供代码直接访问的 va（Virtual Address）。
- buf_dev_mmap 在 rpbuf_dev.c 中使用，供 mmap() 将 buffer 的地址映射到用户空间。

5.4.1.2 在用户空间使用 RPBuf

内核代码 rpbuf_dev.c 会生成 /dev/rpbuf_ctrlX 节点供用户空间程序通过 ioctl() 来使用 RPBuf 框架（每一个 /dev/rpbuf_ctrlX 分别对应每一个 controller）。为方便使用，我们基于此再封装成 librpbuf 库，其 API 位于头文件 librpbuf.h 中：

```
typedef struct __rpbuf_buffer rpbuf_buffer_t;

rpbuf_buffer_t *rpbuf_alloc_buffer(int ctrl_id, const char *name, size_t len);
int rpbuf_free_buffer(rpbuf_buffer_t *buffer);

void *rpbuf_buffer_addr(rpbuf_buffer_t *buffer);
const char *rpbuf_buffer_name(rpbuf_buffer_t *buffer);
size_t rpbuf_buffer_len(rpbuf_buffer_t *buffer);

int rpbuf_buffer_is_available(rpbuf_buffer_t *buffer);

int rpbuf_transmit_buffer(rpbuf_buffer_t *buffer, unsigned int offset, unsigned int data_len);
int rpbuf_receive_buffer(rpbuf_buffer_t *buffer, unsigned int *offset,
        unsigned int *data_len, int timeout_ms);
int rpbuf_install_rx_signal(rpbuf_buffer_t *buffer, sighandler_t handler);
int rpbuf_remove_rx_signal(rpbuf_buffer_t *buffer);
int rpbuf_set_sync_buffer(rpbuf_buffer_t *buffer, uint32_t is_sync);
```

这些 API 与内核头文件 include/linux/rpbuf.h 中的类似，详细用法可参考 librpbuf.h 中的注释和示例 demo。

它们与内核 API 的用法区别，主要在于：

- rpbuf_alloc_buffer() 会阻塞等待 buffer 变为 available，或超时退出（超时的等待时间在 rpbuf_dev.c 中用宏 RPBUF_DEV_WAIT_TIMEOUT_MSEC 定义）。
- rpbuf_buffer_addr() 返回的是执行 mmap() 后将 buffer 地址映射上来的地址，可供用户空间程序直接使用。
- rpbuf_buffer_is_available() 同样是用于检查 buffer 是否为 available，但因为涉及用户态和内核态的切换，并不适合太过频繁地使用。
- 将数据传输到远端同样是使用 rpbuf_transmit_buffer，用法也类似；但因为没有 rx_cb，接收数据则需要应用程序主动调用 rpbuf_receive_buffer() 来判断远端是否传输数据过来（阻塞与否通过 timeout_ms 指定）。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- rpbuf_install_rx_signal() 可以让内核使用异步通知，当内核收到消息传输命令后，会发送一个SIGIO信号给应用层，然后应用层捕获该信号后会调用其处理函数，处理函数通过第二个参数sighandler_t handler进行安装，具体用法可以参考rpbuf_demo 程序章节里 linux 的源码。
- rpbuf_remove_rx_signal() 是让应用捕获到SIGIO信号后执行系统默认操作，不再调用自定义安装的处理函数，与rpbuf_install_rx_signal()成对使用。
- rpbuf_set_sync_buffer()的作用是设置为同步传输，每次调用完rpbuf_transmit_buffer()都会阻塞，直到收到一个消息同步命令之后退出。

⚠ 注意

在应用程序两次调用 rpbuf_receive_buffer() 之间，如果远端传输了两次或以上的数据过来，rpbuf_receive_buffer() 只能拿到最近一次的 offset 和 data_len，因为底层没有缓冲区来保存历史值。这种情况在系统繁忙时比较容易出现，需要应用程序调用rpbuf_set_sync_buffer()设置同步方式来避免覆盖旧数据。

5.4.2 FreeRTOS

FreeRTOS 的 RPBuf 代码结构与 Linux 的类似，但因为不区分内核空间和用户空间，所以不需要 rpbuf_dev 和 librpbuf 这两部分功能的代码。

```
.
├── components
│   ├── aw
│   │   └── rpbuf
│   │       ├── rpbuf_common.h
│   │       ├── rpbuf_controller.c
│   │       ├── rpbuf_core.c
│   │       ├── rpbuf_internal.h
│   │       └── rpbuf_service_rpmsg.c
│   └── thirdparty
│       └── rpbuf_demo/
├── include
│   └── rpbuf.h
└── projects
    └── <project_dir>
        └── src
            └── rpbuf.c      // 在不同 project 中的 rpbuf 差异化配置
```

RPBuf 对外的 API 都位于头文件 include/rpbuf.h 中，大部分的用法与 Linux 的内核 API 类似。

区别较大的主要为用于初始化的 API。FreeRTOS 没有 Linux 那样的设备驱动模型，所以是直接调用以下 API 进行初始化，这些步骤一般放在 projects/<project_dir>/src/rpbuf.c 中进行。

```
/* <rpbuf.h> */

/**
 * rpbuf_init_service - Init service
 * @id: service id
 * @token: used to link with controller
 * @attached_op: attached operation for rpbuf service. What it does depends on
 *               the rpbuf_init_service() implementation.
 * @attached_priv: private data used for attached_op().
```

```
 *
 * What @id and @token means is defined by the backend implementation of
 * rpbuf_init_service.
 *
 * Return pointer to rpbuf service on success, or NULL on failure.
 */
struct rpbuf_service *rpbuf_init_service(int id, void *token,
                        rpbuf_service_attached_op_t attached_op,
                        void *attached_priv);

/**
 * rpbuf_deinit_service - The opposite operation of rpbuf_init_service
 * @service: rpbuf service
 */
void rpbuf_deinit_service(struct rpbuf_service *service);

/**
 * rpbuf_init_controller - Init controller
 * @id: controller id
 * @token: used to link with service
 * @role: MASTER or SLAVE
 * @ops: user-specific rpbuf controller operations
 * @priv: user-specific private data
 *
 * What @id and @token means is defined by the backend implementation of
 * rpbuf_init_controller
 *
 * Return pointer to rpbuf controller on success, or NULL on failure.
 */
struct rpbuf_controller *rpbuf_init_controller(int id, void *token, enum rpbuf_role role,
                        const struct rpbuf_controller_ops *ops,
                        void *priv);

/*
 * rpbuf_deinit_controller - The opposite operation of rpbuf_init_controller
 * @controller: rpbuf controller
 */
void rpbuf_deinit_controller(struct rpbuf_controller* controller);
```

5.5 rpbuf_demo 程序

此应用主要是用来展示 rpbuf 的创建，释放，接收和发送等基本功能的应用，每次执行-s 命令发送一条消息，linux 接收部分用的是异步通知机制。

5.5.1 代码路径

5.5.1.1 linux

```
package/allwinner/rpbuf/demo/main.c
```

5.5.1.2 FreeRTOS

```
lichee/dsp/freertos/components/thirdparty/rpbuf_demo/rpbuf_demo.c
```

5.5.2 编译

linux 需要在 make menuconfig 中选上：

```
Allwinner --->
  RPBuf --->
    <*> rpbuf_demo        # CONFIG_PACKAGE_rpbuf_demo
```

DSP 需要在 make menuconfig 中选上：

```
Components options --->
  Thirdparty components --->
    [*] RPBuf demo        # CONFIG_COMPONENTS_RPBUF_DEMO
```

重新编译烧录固件后，在控制台中执行 help 命令可看到增加了 rpbuf_demo 命令。

5.5.3 基本使用

linux 下的 rpbuf_demo 的 help 信息如下：

```
USAGE:
 rpbuf_demo [OPTIONS]

OPTIONS:
 -s MESSAGE  : send MESSAGE
 -d time     : delay free buffer (default: 100 ms)
 -r          : receive messages
 -t time     : specifies the time of receive messagess, unit:ms
 -I ID       : specify rpbuf ctrl ID (default: 0)
 -N NAME     : specify buffer name (default: "rpbuf_demo")
 -L LENGTH   : specify buffer length (default: 32 bytes)
 -O OFFSET   : specify offset in buffer to store data (default: 0)

e.g.
 rpbuf_demo -s "hello" : send string "hello"
 rpbuf_demo -r         : receive data forever
 rpbuf_demo -r -t 1000  : receive data 1000 msecond
```

dsp 下的 rpbuf_demo 的 help 信息如下：

```
USAGE:
 rpbuf_demo [OPTIONS]
OPTIONS:
 -h        : print help message
 -c        : create buffer
 -d        : destory buffer
 -s STRING  : copy STRING to buffer and send to remote
 -l        : list created buffers
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
 -I ID     : specify controller ID (default: 0)
 -N NAME    : specify buffer name (default: "rpbuf_demo")
 -L LENGTH   : specify buffer length (default: 32 bytes)
 -O OFFSET   : specify offset in buffer to store data (default: 0)

e.g.
 First, create a buffer (its name and length should match that of remote rpbuf buffer):
    rpbuf_buffer -N "xxx" -L LENGTH -c
 Then if remote sends data to it, the buffer callback will be called.

 We can send data (e.g. "hello") to remote:
    rpbuf_demo -N "xxx" -s "hello"

 If this buffer is no longer in use, destroy it:
    rpbuf_demo -N "xxx" -d
```

首先，DSP 端使用 -c 选项创建 rpbuf（可使用 -N指定名字，使用-I指定远端通信核的 ID、使用-L 指定长度）。通过 -l 选项可列出之前创建过的 rpbuf。

下表列出本地核中各 ID 分别对应哪个远端核：

本地核	ID：0	ID：1
CPU	DSP0	DSP1
DSP0	CPU	DSP1
DSP1	CPU	DSP0

DSP 端创建了 rpbuf 之后，如果远端有消息发送过来，其回调函数会被调用到，将收到的数据打印出来。

使用 -s 选项可将某个字符串发送到远端。

如果不再使用该 rpbuf，使用 -d 参数进行销毁。

下面演示 rpbuf_demo 最简单的使用方式：

```
linux与DSP0通信：

 （ID默认是0，NAME默认rpbuf_demo）

1、DSP0输入rpbuf_demo -c

2、linux输入rpbuf_demo -s "hello"，DSP0打印输出

3、linux输入rpbuf_demo -r

4、DSP0输入rpbuf_demo -s "123"，linux打印输出
```

```
linux与DSP1通信：

 （ID默认是0，NAME默认rpbuf_demo）

1、DSP1输入rpbuf_demo -c

2、linux输入rpbuf_demo -I 1 -s "hello"，DSP1打印输出
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3、linux输入rpbuf_demo -I 1 -r

4、DSP1输入rpbuf_demo -s "123"，linux打印输出

DSP0与DSP1通信：

（因为不能创建同名buffer，假如前面已经创建了名字为rpbuf_demo的buffer，则需要-N指定名字）

1、DSP0输入rpbuf_demo -I 1 -N "rpbuf_dsp_demo" -c

2、DSP1输入rpbuf_demo -I 1 -N "rpbuf_dsp_demo" -c

3、DSP0输入rpbuf_demo -I 1 -N "rpbuf_dsp_demo" -s "hellp"，DSP1打印输出

4、DSP1输入rpbuf_demo -I 1 -N "rpbuf_dsp_demo" -s "hellp"，DSP0打印输出

技巧

两个核申请创建 rpbuf 时，必须指定名字和长度一致，这样才会访问到同一块内存区域。

5.5.4 rpbuf_test 程序

5.5.4.1 代码路径

此应用是在 rpbuf_demo 的基础上修改的，每次发送会生成一个随机数和 md5 的值，接收端会进行 md5 的校验，可以自定义发送消息的数量和时间间隔，linux 接收部分用的是轮询机制。

5.5.4.1.1 linux

```
package/allwinner/rpbuf/test/main.c
```

5.5.4.1.2 FreeRTOS

```
lichee/dsp/freertos/components/thirdparty/rpbuf_demo/rpbuf_test.c
```

5.5.4.2 编译

linux 需要在 make menuconfig 中选上：

```
Allwinner --->
  RPBuf --->
    <*> rpbuf_test        # CONFIG_PACKAGE_rpbuf_test
```

DSP 需要在 make menuconfig 中选上：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
Components options --->
  Thirdparty components --->
    [*] RPBuf demo        # CONFIG_COMPONENTS_RPBUF_DEMO
```

重新编译烧录固件后，在控制台中执行 help 命令可看到增加了 rpbuf_test 命令。

5.5.4.3 基本使用

linux 下的 rpbuf_test 的 help 信息如下：

```
USAGE:
  rpbuf_test [OPTIONS]

OPTIONS:
  -d time   : set data sending interval (default: 100 ms)
  -s        : send test messages
  -c        : send count (default: 10)
  -r        : receive messages
  -t time   : specifies the time of receive messagess, unit:ms
  -a        : sync transmit
  -v        : verbosely print check result
  -I ID     : specify rpbuf ctrl ID (default: 0)
  -N NAME   : specify buffer name (default: "rpbuf_test")
  -L LENGTH : specify buffer length (default: 32 bytes)

e.g.
  rpbuf_test -L 0x1000 -c 10 -s -a : send 10 test data, size=0x1000
  rpbuf_test -L 0x1000 -c -1 -s -a : send test data all the time, size=0x1000
  rpbuf_test -L 0x1000 -r          : receive test data forever, size=0x1000
  rpbuf_test -L 0x1000 -r -t 1000  : receive test data 1 second, size=0x1000
```

dsp 下的 rpbuf_test 的 help 信息如下：

```
USAGE:
  rpbuf_test [OPTIONS]
OPTIONS:
  -h        : print help message
  -c        : create buffer
  -d        : destory buffer
  -s        : send test messagese
  -l        : list created buffers
  -a        : sync transmit
  -v        : verbosely print check result
  -C        : send count (default: 1,always send: -1)
  -I ID     : specify controller ID (default: 0)
  -N NAME   : specify buffer name (default: "rpbuf_test")
  -D TIME   : set data sending interval (default: 100 ms)
  -L LENGTH : specify buffer length (default: 32 bytes)

e.g.
  First, create a buffer (its name and length should match that of remote rpbuf buffer):
    rpbuf_test -N "xxx" -L LENGTH -c -a
  Then if remote sends data to it, the buffer callback will be called.

  We can send 100 test data to remote:
    rpbuf_test -N "xxx" -C 100 -s
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
We can always send test data to remote, interval time 5ms:
  rpbuf_test -N "xxx" -C -1 -D 5 -s

If this buffer is no longer in use, destroy it:
  rpbuf_test -N "xxx" -d
```

首先，DSP 端使用 -c 选项创建 rpbuf（可使用 -N指定名字，使用-I指定远端通信核的 ID、使用-L 指定长度）。通过 -l 选项可列出之前创建过的 rpbuf。

下表列出本地核中各 ID 分别对应哪个远端核：

本地核	ID：0	ID：1
CPU	DSP0	DSP1
DSP0	CPU	DSP1
DSP1	CPU	DSP0

DSP 端创建了 rpbuf 之后，如果远端有消息发送过来，DSP 定义的回调函数会被调用到，将收到的数据打印出来。

使用 -s 选项可创建发送线程，发送随机生成的数据并在接收端中校验。

如果不再使用该 rpbuf，使用 -d 参数进行销毁。

下面演示 rpbuf_test 最简单的使用方式：

```
linux与DSP0通信：

（ID默认是0，NAME默认rpbuf_test）

linux发dsp0收：
1、dsp0执行rpbuf_test -I 0 -L 1024 -N "cpu_to_dsp0" -c
2、linux执行rpbuf_test -I 0 -L 1024 -N "cpu_to_dsp0" -s -c -1 -a &

dsp0发linux收：
1、dsp0执行rpbuf_test -I 0 -L 1024 -N "dsp0_to_cpu" -c -a
2、linux执行rpbuf_test -I 0 -L 1024 -N "dsp0_to_cpu" -r &
3、dsp0执行rpbuf_test -N "dsp0_to_cpu" -C -1 -s
```

```
linux与DSP1通信：

（ID默认是0，NAME默认rpbuf_test）

linux发dsp1收：
1、dsp1执行rpbuf_test -I 0 -L 1024 -N "cpu_to_dsp1" -c
2、linux执行rpbuf_test -I 1 -L 1024 -N "cpu_to_dsp1" -s -c -1 -a&

dsp1发linux收：
1、dsp1执行rpbuf_test -I 0 -L 1024 -N "dsp1_to_cpu" -c -a
2、linux执行rpbuf_test -I 1 -L 1024 -N "dsp1_to_cpu" -r &
3、dsp1执行rpbuf_test -I 0 -N "dsp1_to_cpu" -C -1 -s
```

```
DSP0与DSP1通信：

（ID默认是0，NAME默认rpbuf_test）
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
dsp0发dsp1收：
1、dsp1执行rpbuf_test -I 1 -L 1024 -N "dsp0_to_dsp1" -c
2、dsp0执行rpbuf_test -I 1 -L 1024 -N "dsp0_to_dsp1" -c -a
3、dsp0执行rpbuf_test -N "dsp0_to_dsp1" -C -1 -s

dsp1发dsp0收：
1、dsp0执行rpbuf_test -I 1 -L 1024 -N "dsp1_to_dsp0" -c
2、dsp1执行rpbuf_test -I 1 -L 1024 -N "dsp1_to_dsp0" -c -a
3、dsp1执行rpbuf_test -N "dsp1_to_dsp0" -C -1 -s
```

技巧

两个核申请创建 rpbuf 时，必须指定名字和长度一致，这样才会访问到同一块内存区域。

广州星翼电子科技有限公司zdyzlinux

著作权声明

版权所有 ©2023 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

ALLWINNER、ALLWINNER 全志科技、全志科技、ALLWINNER（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。